

Firebase

+

Firestore

+

Firebase Cloud Firestore

+

Kotlin Flow

+

Android App Development

+

How To Make Your Android App Extra Reactive (feat. Firebase Cloud Firestore)

7 min read · May 21, 2024



Emmanuel Muturia™

Follow



Listen



Share

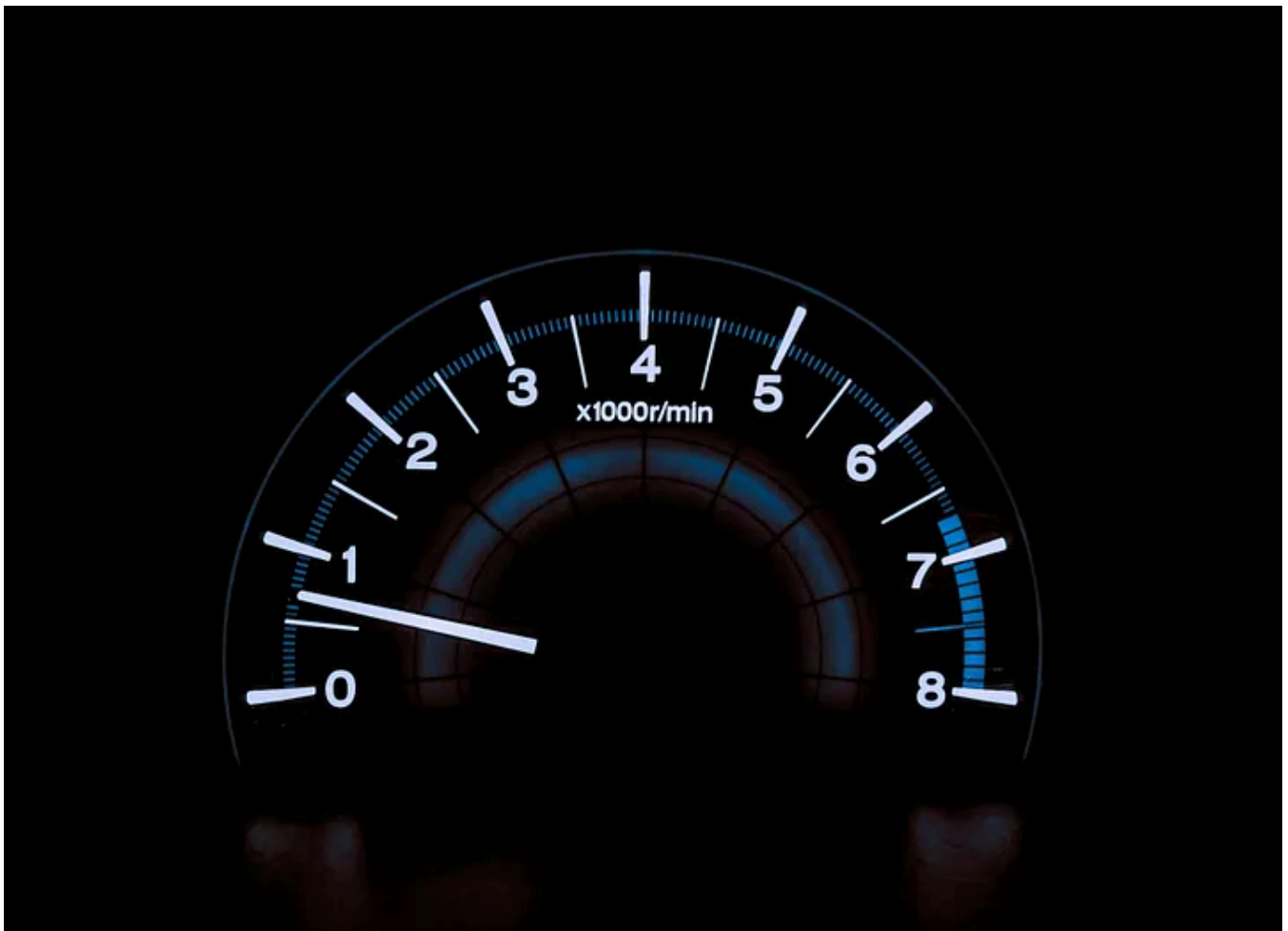


Photo by [Chris Liverani](#) on [Unsplash](#)

INTRO

You have built that lovely Android app of yours, shipped it, and are enjoying the fruits of your labour when it suddenly hits you. Something just needs to be quite right. The app needs **The Special Sauce**. “What could it be, Emmanuel? What is **The Special Sauce**? Is it the UI? The UX? The Security Metrics? Tell me!”

BACKGROUND

This is the part where you open your existing Android Studio project that is already connected to Firebase and head over to your implementation of Firebase Cloud Firestore. This article will guide you on getting real-time updates from your Firebase Cloud Firestore collection with just a few lines of code. If you have been a recent visitor of the [Firebase Cloud Firestore Documentation for Android](#), then you know how open-ended it is to fetch data from your collection. Besides, this same attribute might make it hard for you to know what method suits you best, right? Allow me to alleviate your troubles by presenting to you a simple solution that will especially suit you if you need your Android app to display and update data in real-time at any given time. This is: **How To Make Your Android App Extra Reactive feat. Firebase Cloud Firestore...**



DISCLAIMER

Before we continue, I would like to mention the following:

1. This methodology is not set in stone. The best way to go about it is to further explore and find **your** best way of achieving the same result...
2. This article does not cover the steps needed to integrate your app with Firebase Cloud Firestore. If you need to learn that, please head over to [The Firebase Cloud Firestore Documentation](#) and then quickly come back so that you may not get left behind. I believe in you :)

3. These concepts (except for Snapshot Listeners as they are specific to Firebase) are transferrable to other Remote Data Sources such as REST APIs but for the sake of simplicity, we are going to use Firebase Cloud Firestore...

That is all. Now, let us learn how to make our Android apps even more reactive...



CALLBACKFLOW AND SNAPSHOT LISTENERS

Whoa whoa whoa before you run off in fear, please understand that these two concepts are easy to understand and by the end of this article, you will not only have a solid understanding of them but will also have to restrain yourself from using them as they are that good. Before we can dive into code, let us define them, shall we:

1) CALLBACK FLOW

If you have been practising Android (Kotlin) for a while, then you are well aware of this one topic that you can never flee whenever you talk about Reactive, Asynchronous, or even Concurrent ways of programming: **Kotlin Coroutines**. Kotlin Coroutines allow us to collect and handle streams of data asynchronously through **Flows**. Of course, you know this well, right? When collecting collections (pardon my

pun), it is a common practice to use the **flow {}**, **collect {}**, and **emit()** operators among others to collect the data asynchronously thus making our Android apps reactive. However, there is a catch. Using them alone does not guarantee that your app will automatically reflect any changes made on our remote data source, in this case, Firebase Cloud Firestore. Will it not be incredible if we could have our data automatically updated without having us refresh our screens and stuff?



SNAPSHOT LISTENER

As previously mentioned, there are several ways of reading data from our Firebase Cloud Firestore collections. Most of them will only achieve this by fetching your data once which is only when you request it. However, there is one sneaky way of ensuring that our Android app listens to (pun intended as you will see shortly) any changes to our remote data and reactively makes changes in real time. That way is through the use of **Snapshot Listeners**. Snapshot Listeners are real-time queries that ensure our Android apps are notified of any slight changes to our remote data in a low-latency fashion. Think of them as messengers that instantly send updates to our Android apps in case of any slight changes...



THE MARRIAGE OF CALLBACK FLOW AND SNAPSHOT LISTENERS

No, you will not need a suit or gown for this as we are not conducting a wedding. Rather, this is the part you have been waiting for. How do we use both `callbackFlow` and Snapshot Listeners to make our Android apps reactive in real time? This is how:

```
class YourRepositoryImplementation(
    private val firebaseFirestore: FirebaseFirestore
) : YourRepository {

    override fun getYourData(): Flow<List<YourModel>> = callbackFlow {

        firebaseFirestore.collection("YourCollection").addSnapshotListener { va

            if (error != null) {
                return@addSnapshotListener
            }

            if (value != null && !value.isEmpty) {
                trySend(element = value.toObject<YourModel>()).isSuccess
            }

        }

        awaitClose { this.cancel(message = "Error encountered. Please try again

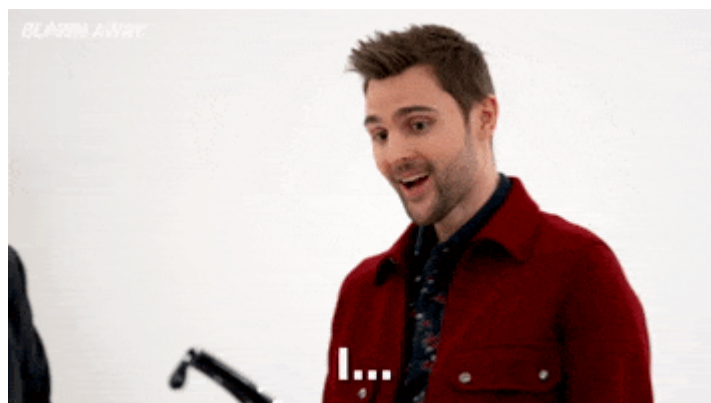
    }

}
```

That is all you need. Here is what is happening:

- This is a **Repository-Pattern Implementation** that defines a function that was defined in a fictional `YourRepository` Interface. You do not have to follow this structure as the code inside the `YourRepositoryImplementation` class remains the same regardless of the implementation...

- Assuming that we are getting a list of data from Firebase Cloud Firestore, we are wrapping it inside `Flow<T>` to collect it asynchronously and ensure that we always have the latest data when we use `collect {}` as you will see. We then return a `callbackFlow {}` which converts our data stream which is in the form of a Callback into a **Cold Flow**, which simply refers to a data stream that does not start producing values until another entity starts collecting them. This is useful because it means that we conserve limited computing resources while also ensuring that we collect the latest data, in this case, any changes made to our data stream...
- We access our data using `firebaseFirestore.collection("NameofCollection")` and use `addSnapshotListener { value, error -> }` to add a Snapshot Listener to our data stream so that it can observe or “listen” to any changes made to the data stream...
- The first code block inside is used for error handling...
- The second code block uses `trySend {}` to add the data stream to the Channel used in this operation and return the result if it satisfies the conditions of not being null and not being empty...
- The final line of code suspends the current operation until the Channel is either closed or cancelled and invokes the given code block inside it before resuming the coroutine. This means that the whole operation is allowed to run until it is explicitly cancelled or closed at which point it outputs the message indicated...



VIEWMODEL IMPLEMENTATION

To implement this in your ViewModel and display the data in your Compose UI, follow the following steps. Please note that this may differ especially if you are using

a different Architectural Pattern other than MVVM but the concept remains the same:

- Ensure that you have an instance of `YourRepository` in the constructor...
- Declare the two state variables to be used to display the data stream in your UI: **One should be mutable and the other immutable and read-only. This is because the mutable state variable will be used to update the state based on the data collected while the immutable state variable will be used to monitor the updated state and inform the Compose UI accordingly...**
- Define a function and call your Repository Function inside it to collect your data stream via the `YourRepository` property you defined...
- Define a state variable in your Compose UI to observe any changes to your data stream and update your UI accordingly...
- Putting these together, you should have:

```
class YourViewModel (
    private val yourRepository: YourRepository
): ViewModel() {

    private var _yourData: MutableStateFlow<CarizmaState<List<YourModel>>> = MutableStateFlow()
    val yourData: StateFlow<CarizmaState<List<YourModel>>> = _yourData.asStateFlow()

    init {
        getData()
    }

    private fun getData() {
        viewModelScope.launch {
            yourRepository.getData().collect { data ->
                _yourData.value = data
            }
        }
    }
}
```

Finally, you will of course observe the state changes as follows based on the final step in your Compose UI:

```
// You will define your ViewModel here depending on which way you use for Deper  
val yourState by yourViewModel.yourData.collectAsState()  
// You can also use .collectAsStateWithLifecycle() if you want to collect state
```



CONCLUSION

Now that you know how to make your Android apps more reactive, go on, add this, and watch your users enjoy the power of real-time data. Easy there, do not thank me yet for there is more for us to learn out there but this will do for today. All the best in your work and I wish you nothing but success. Cheers!



REFERENCES

callbackFlow. (n.d.). Kotlinlang.org. Retrieved May 21, 2024, from [https://kotlinlang.org/api/kotlinx-coroutines/](https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-)

Open in app ↗

Sign up

Sign in

Medium

Search



Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Get realtime updates with Cloud Firestore. (n.d.). Firebase. Retrieved May 21, 2024, from <https://firebase.google.com/docs/firestore/query-data/listen#kotlin+ktx>

Understand real-time queries at scale | Firestore. (n.d.). Firebase. Retrieved May 21, 2024, from https://firebase.google.com/docs/firestore/real-time_queries_at_scale#:~:text=Real%2Dtime%20queries%2C%20also%20called

Firebase

+

Firestore

+

Firebase Cloud Firestore

+

Kotlin Flow

+

Android App Development

+



Follow

Written by Emmanuel Mutura™

48 followers · 1 following

Hi, my name is Emmanuel Mutura™, and I help fight Crime on The Internet through my practice in Cyber Security Engineering...

No responses yet



Write a response

What are your thoughts?

